COS214 Prac 4 - TaskForge

Team Details

Ishmael Ngwasheng - u24642542

Ngeletshedzo Mutwanamba - u25039769

Takunda Mugwagwa - u24667341

Task 1

System Description

TaskForge models a film production's workflow: an Act contains Sequences, which contain Scenes, which contain individual Shots. The system needs to treat single Shots and whole groups uniformly, traverse the hierarchy in different ways (full vs. urgent-only), track each Shot's lifecycle (Scripted → Shooting → Approved, with reshoots looping back), and attach optional, stackable extras like insurance or priority handling. TaskForge solves this with Composite (the Act/Sequence/Scene/Shot hierarchy), Iterator (full and urgent traversals), State (a Shot's lifecycle), and Decorator (stackable runtime responsibilities), combined into one coherent system rather than four separate demos.

Design Rationale

Composite: ProductionUnit is the shared interface for Shot (leaf) and ProductionGroup/Act/Sequence/Scene (composites), so client code treats both uniformly. Groups own and delete their children.

Iterator: ProductionGroup never exposes its internal vector — only createFullIterator()/createUrgentIterator(). FullIterator does a complete pre-order traversal; UrgentIterator wraps its own FullIterator and filters for urgency, so two independent traversals can run over the same tree at once.

State: A Shot's behaviour (urgency, valid/invalid transitions) genuinely changes by lifecycle stage, so that logic lives in ShotState subclasses rather than conditionals in Shot. Shot owns its current state and deletes the old one on every transition.

Decorator: Responsibilities like insurance or priority are optional and combinable, so ProductionDecorator wraps any ProductionUnit and lets them stack at runtime, without exploding the class hierarchy. Each decorator owns and deletes what it wraps.

GoF Participant Mapping

Composite

GoF Role	**Class**
Component	ProductionUnit
Composite	ProductionGroup (abstract), Sequence, Act, Scene (concrete)
Leaf	Shot

Iterator

GoF Role	**Class**
Aggregrate	ProductionUnit
ConcreteAggregate	ProductionGroup (abstract), Sequence, Act, Scene (concrete)
Iterator	ProductionIterator
ConcreteIterator	FullIterator, UrgentIterator

State

GoF Role	**Class**
Context	Shot
State	ShotState
ConcreteState	Scripted, Approved, Shooting

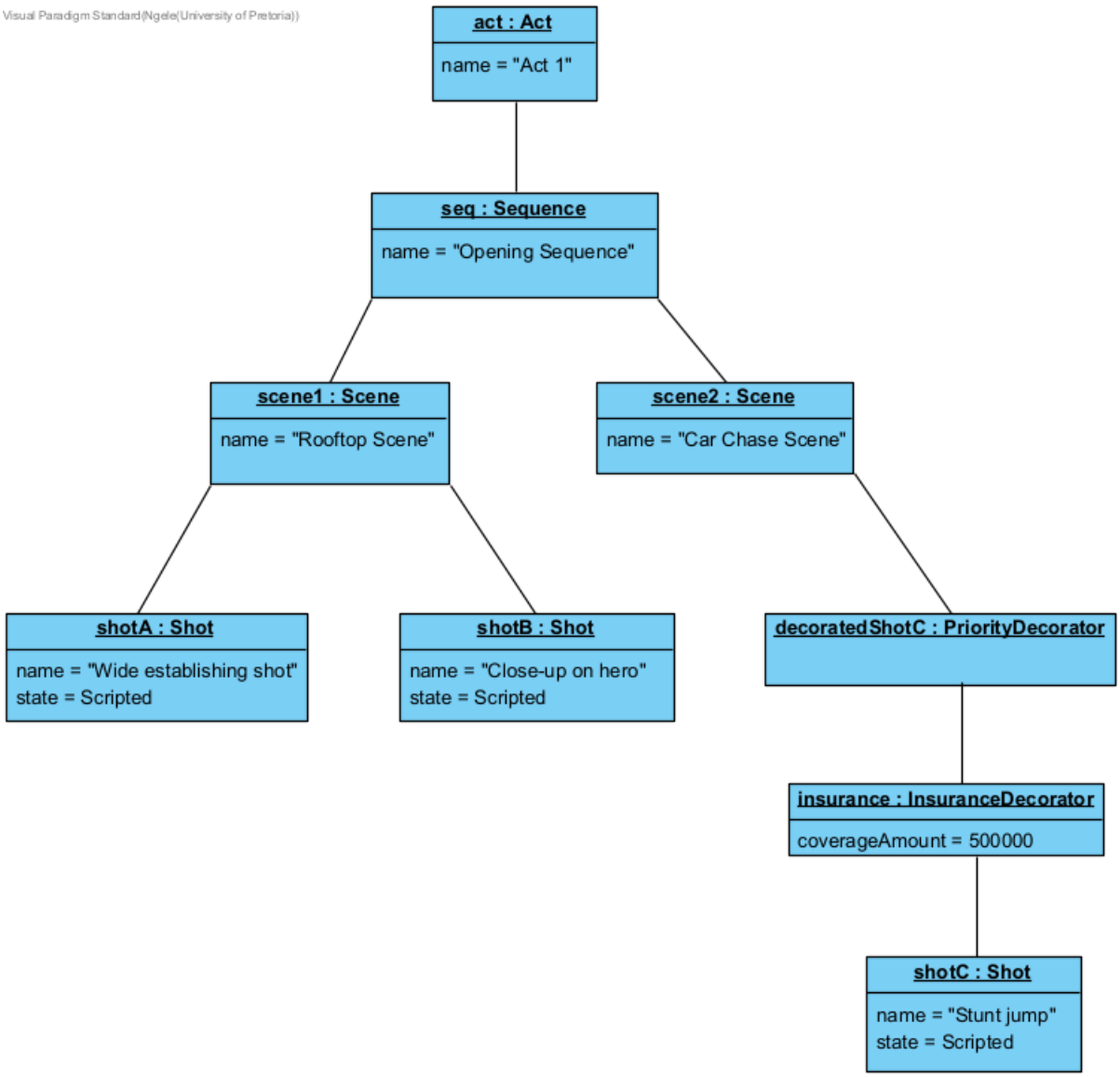
Decorator

GoF Role	**Class**
Component	ProductionUnit
ConcreteComponent	ProductionGroup Shot
Decorator	ProductionDecorator

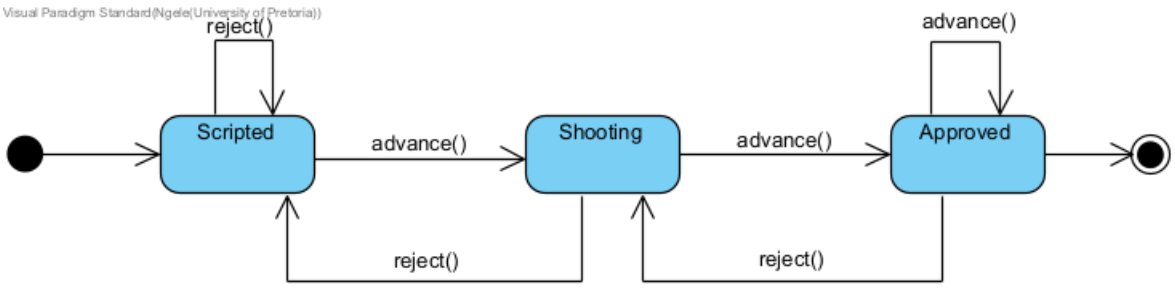
ConcreteDecorator	PriorityDecorator, InsuranceDecorator

Task 4 Diagrams

Object Diagram

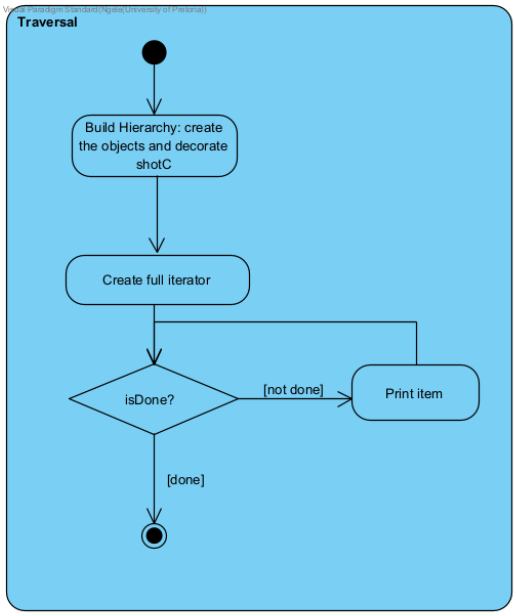


State Diagram

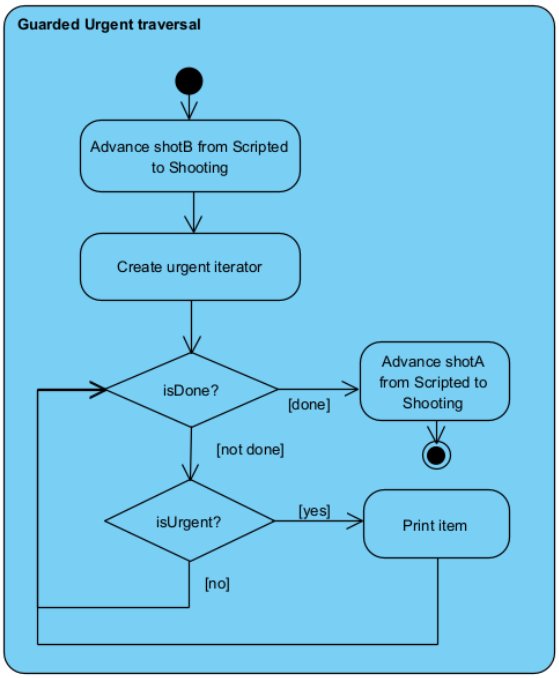


Activity Diagrams

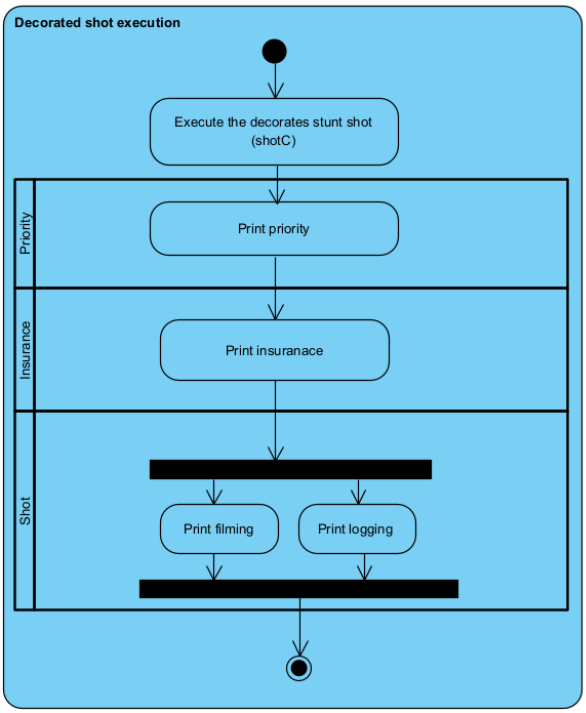
1.



2.



3.



Traversal Maintenance

"Each iterator builds its own list once, when it's created — it walks the tree at that moment and stores either every unit, or just the urgent ones, along with a position index. After that, it just steps through its own list and never looks at the tree again. So if something changes afterward — like a shot moving or advancing state — the iterator won't see it, since it already took its snapshot. To get the updated view, you just make a new iterator.

We actually show this directly: shotA advances partway through the demo, and we create a new UrgentIterator afterward to prove it now picks up shotA, while the older iterator wouldn't have. We went with this snapshot approach instead of reading live from the tree mainly for safety — a live iterator risks pointing at something that's already been deleted if the tree changes mid-traversal.

Task 5

GDB Investigation

We put a breakpoint on Shot::advance() and ran the program. It stopped right before state->advance(this) was called, on the shot called "Close-up on Hero". At that point, this->state->getName() returned "Scripted". After stepping past that one line with next, we checked again and it returned "Shooting".

This shows that Scripted::advance() is actually replacing the shot's state pointer with a new Shooting object, not just changing some internal flag or label. It's real proof that our State pattern is doing what it's supposed to do at runtime.

```
Breakpoint 1, Shot::advance (this=0x5555555774b0) at Shot.cpp:24
24          state->advance(this);
(gdb) print this->name
$3 = "Close-up on Hero"
(gdb) print this->state->getName()
$4 = "Scripted"
(gdb) next
25      }
(gdb) print this->state->getName()
$5 = "Shooting"
(gdb)
```

Bug Investigation

Symptom: When we tried to compile and run our decorators, we got a compiler error saying that we *"cannot declare variable to be of abstract type 'InsuranceDecorator'"* on the line in main.cpp where we create the decorated shot.

Cause: ProductionUnit declares getName(), getStatus(), and isUrgent() as const pure virtual functions. In our ProductionDecorator class, we'd written these same three methods without the const keyword. Since const is actually part of a function's signature in C++, this meant our versions weren't overriding ProductionUnit's pure virtuals at all — they were just separate, unrelated declarations. This left the real pure virtual functions still unimplemented.

Debugging evidence: The compiler error pointed straight at the line in main.cpp where we do new InsuranceDecorator(...), and listed the specific pure virtual functions that were still unimplemented. Comparing that list against ProductionUnit.h showed all three were declared const there, but our ProductionDecorator.h versions weren't.

Correction: We added const to all three method declarations in ProductionDecorator.h and gave each one a default implementation that just forwards to the wrapped object

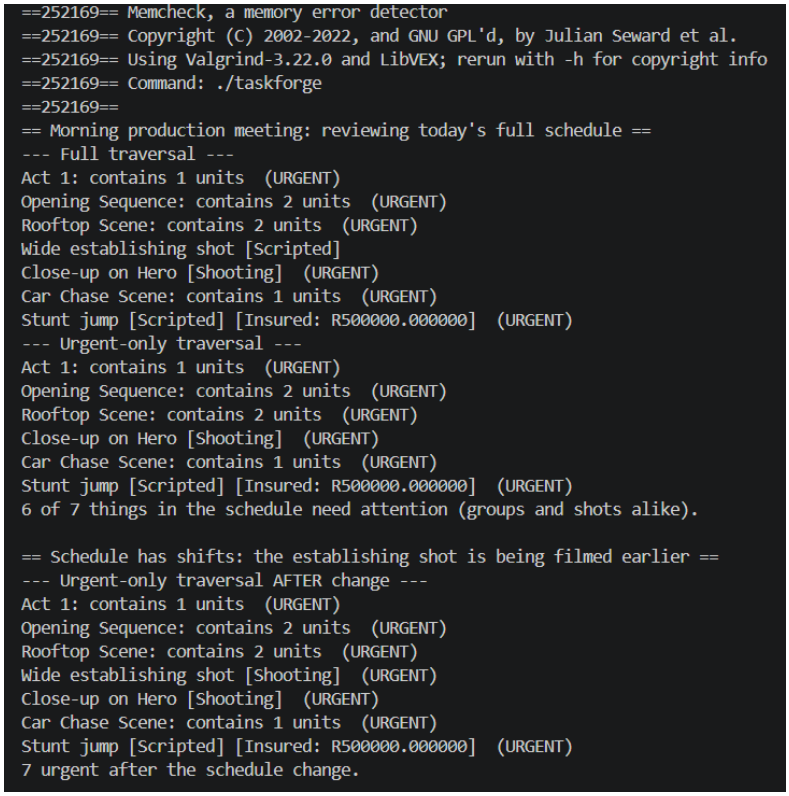
(e.g. return wrapped->getName();). This meant our concrete decorators (InsuranceDecorator, PriorityDecorator) only needed to override the specific methods where their behaviour actually differs, instead of reimplementing everything.

Valgrind Evidence

We ran Valgrind with full leak checking on the finished program:

valgrind --leak-check=full --show-leak-kinds=all ./taskforge

The heap summary showed 75 allocations and 75 frees, with 0 bytes in use at exit and no errors reported. This confirms that every object we created with new — including the composite tree, the shot states, the decorators, and all three iterators — was correctly cleaned up, mostly through the ownership chain we built into our destructors (ProductionGroup deleting its children, Shot deleting its current state, UnitDecorator deleting whatever it wraps), plus the explicit delete calls on our iterators in main.cpp.



```
== Wrapping the day: the insured stunt jump goes ahead ==
[Priority] Escalating "Stunt jump" ahead of the queue
[Insurance] Verifying R500000 coverage for "Stunt jump"
Filming shot "Stunt jump" (Scripted)
Logging shot "Stunt jump" in the production diary
==252169==
==252169== HEAP SUMMARY:
==252169==     in use at exit: 0 bytes in 0 blocks
==252169==   total heap usage: 75 allocs, 75 frees, 79,029 bytes allocated
==252169==
==252169== All heap blocks were freed -- no leaks are possible
==252169==
==252169== For lists of detected and suppressed errors, rerun with: -s
==252169== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

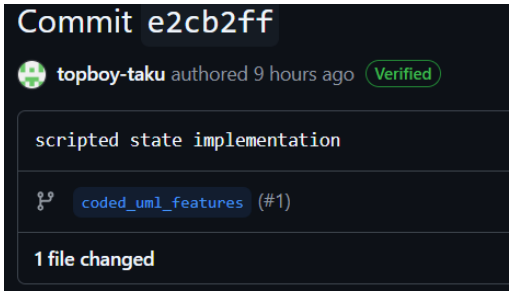
Task 6

Github Workflow

https://github.com/ishmael-97/COS214_Prac_4

Our team split work by pattern responsibility and used feature branches to keep each person's work isolated while building, merging into main once each part was tested individually. This let development happen in parallel: the Composite/Iterator work, the State pattern, and the Decorator classes were all built without one incomplete piece blocking anyone else's build. We used the feature branches coded_uml_features and document-and-diagrams to keep our work isolated while building before merging into main. This let us work on different parts of the design at the same time without one person's incomplete code blocking someone else's build.

A few commits that show this collaboration:



Task 7

Team Contribution

Ishmael Ngwasheng: Implemented the Composite pattern and the Iterator pattern ; set up the Docker environment and managed the GitHub repository for Task 6.

Takunda Mugwagwa: Implemented the State pattern (ShotState and its concrete states) and the Decorator pattern (ProductionDecorator, InsuranceDecorator, PriorityDecorator); wrote main.cpp; wrote the design rationale and system description.

Ngeletshedzo Mutwanamba: Built the UML class diagram and the Task 4 diagram portfolio (object diagram, state diagram, activity diagrams); captured the GDB and Valgrind evidence for Task 5; structured and compiled the full PDF submission.